

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Аргументы

Парсинг командной строки с argparse

Модуль `argparse` парсит аргументы. Использование этого модуля даёт пользователям программы возможность передавать аргументы во время выполнения программы, а также выводить справку по командам.

Модуль включён в стандартную библиотеку пайтона и установки не требует. Сначала нужно импортировать модуль и создать объект парсера.

```
import argparse
```

```
parser = argparse.ArgumentParser()
```

Можно при желании добавить описание скрипта (description).

Метод `parse_args()` считывает аргументы командной строки, по умолчанию уже есть один опциональный аргумент `-h` или `-help`. При желании отключить ключ `-h` можно дописав аргумент `add_help=False`. Чтобы посмотреть вывод справки аргументов, воспользуемся методом `print_help()`.

```
import argparse

parser = argparse.ArgumentParser(description='Description of your program')
args = parser.parse_args()
parser.print_help()
```

```
all x
usage: lesson.py [-h]

Description of your program

optional arguments:
  -h, --help  show this help message and exit
```

Точно такой же вывод мы получим, если запустим скрипт с аргументом `-help`.

```
(venv) root@exp:~/PycharmProjects/education# python lesson.py --help
usage: lesson.py [-h]

Description of your program

optional arguments:
  -h, --help  show this help message and exit
```

По-умолчанию `parse_args()` анализирует все добавленные в наш объект аргументы и генерирует строку `usage` с учетом синтаксиса и типа этих аргументов.

Значение `usage` можно изменить при создании объекта парсера, определив параметр `usage`.

```
import argparse

parser = argparse.ArgumentParser(description='Description of your program',
                                usage='Script options')

args = parser.parse_args()
parser.print_help()
```

```
all x
usage: Script options

Description of your program

optional arguments:
  -h, --help  show this help message and exit
```

Аргументы добавляются вызовом метода `add_argument()`. Аргументы бывают позиционные (обязательные) и опциональные (необязательные).

Опциональные аргументы отличаются от позиционных наличием префикса. По умолчанию в созданном экземпляре парсера префикс дефис. Но можно установить любой, или даже несколько разных, передав в качестве параметра парсеру `prefix_chars='+'`.

```
import argparse

parser = argparse.ArgumentParser(description='Description of your program',
                                usage='Script options', prefix_chars='+')

args = parser.parse_args()
parser.print_help()
```

```
all x
usage: Script options

Description of your program

optional arguments:
  +h, ++help  show this help message and exit
```

Для добавленных аргументов можно сделать описание через параметр `help`. По умолчанию тип данных аргументов строка, но мы можем изменить тип через параметр `type`.

Сделаем простенькую функцию возведения в степень. Чтобы получить доступ к аргументам, в вызове функции мы будем обращаться к ним через точку.

При вызове справки, мы видим, что добавились наши позиционные аргументы.

```

1  import argparse
2
3  parser = argparse.ArgumentParser(description='Description of your program')
4  parser.add_argument('n', type=int, help='number')
5  parser.add_argument('e', type=int, help='exponentiation')
6  args = parser.parse_args()
7
8
9  def exponent(n, e):
10     print(n**e)
11
12
13     exponent(args.n, args.e)

```

Terminal: Local x +

```

usage: lesson.py [-h] n e

Description of your program

positional arguments:
  n          number
  e          exponentiation

```

Запустив программу со значениями аргументов 3 и 4, получаем результат 81, всё работает как задумано.

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4
81

```

Опциональные аргументы (ключевые, именованные), пишутся с одним префиксом для короткого названия, и с двумя для более длинной версии названия (псевдоним аргумента, это необязательная опция). Опциональные аргументы не являются обязательными, то есть их может быть довольно много, и при запуске программы, мы можем выбирать ту или иную опцию/опции.

Добавим один опциональный аргумент с двумя версиями названия, и параметром `default='exponent'` указывающем, что функция 'exponent' будет срабатывать по умолчанию.

Посмотрим, как будет выглядеть справка.


```

6 parser.add_argument('-s', '--sum', help='sum of numbers',
7                     default='exponent')

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py --help
usage: lesson.py [-h] [-s SUM] n e

Description of your program

positional arguments:
  n          number
  e          exponentiation

optional arguments:
  -h, --help            show this help message and exit
  -s SUM, --sum SUM     sum of numbers

```

Сделаем вторую функцию сложения чисел `sum_num`, и добавим условие, при котором будет запускаться та или иная функция. Запускаем скрипт и видим, что без указания опции срабатывает функция возведения в степень.

```

15 def sum_num(n, e):
16     print(n + e)
17
18
19 if args.sum == 'exponent':
20     exponent(args.n, args.e)
21 elif args.sum == 'sum_num':
22     sum_num(args.n, args.e)
23
24 exponent()

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4
81
(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4 -s exponent
81
(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4 -s sum_num
7

```

Пространство имен — это система, которая имеет уникальное имя для каждого объекта в Python. Объект может быть переменной или методом. Сам Python поддерживает пространство имен в форме словаря.

Вывести информацию из пространства имён можно так - `print(args)`

```

6 parser.add_argument('-s', '--sum', help='sum of numbers',
7 args = parser.parse_args()
8 print(args)
9
Terminal: Local x +
(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4
Namespace(e=4, n=3, sum='exponent')
81

```

При использовании другого опционального аргумента, соответственно в пространстве имён будет уже он.

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py 3 4 -s sum_num
Namespace(e=4, n=3, sum='sum_num')
7

```

Действия - параметр `action` указывает парсеру, как необходимо обрабатывать задаваемый аргумент и его значение. По умолчанию, значение аргумента просто сохраняется (`store`) в `namespace` (пространстве имён). То есть, если мы напишем так:

```

1 import argparse
2
3 parser = argparse.ArgumentParser(description='Description of your program')
4 parser.add_argument('-x', action='store')
5 args = parser.parse_args()
6 print(args)

```

```

Terminal: Local x +
(venv) root@exp:~/PycharmProjects/education# python lesson.py
Namespace(x=None)
(venv) root@exp:~/PycharmProjects/education# python lesson.py -x 26
Namespace(x='26')
(venv) root@exp:~/PycharmProjects/education#

```

То никакой разницы не заметим, есть такой параметр или нет, так как `store` уже идёт по-умолчанию.

Если значение аргумента всегда постоянно (константа) - пользователю нет необходимости каждый раз указывать в командной строке это значение, достаточно просто указать аргумент. Чтобы парсер в данном случае корректно отработал и сохранил константу как значение аргумента, `'action'` присваивается значение `store_const` и дополнительно определяется константа `const='значение'`.

```

1 ► import argparse
2
3     parser = argparse.ArgumentParser(description='Description of your program')
4     parser.add_argument('-x', action='store_const', const='3')
5     args = parser.parse_args()
6     print(args)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py
Namespace(x=None)
(venv) root@exp:~/PycharmProjects/education# python lesson.py -x
Namespace(x='3')
(venv) root@exp:~/PycharmProjects/education#

```

Видно, что при передаче программе аргумента `-x`, в namespace сразу попадает значение `3` без его явного указания. Вместе с тем, если аргумент не передаётся, то в namespace значение `None`.

Также есть параметры булева типа `'store_true'` и `'store_false'`. Для `'store_true'`: если аргумент был указан в командной строке - в пространстве имен ему присваивается значение `'True'`, если нет - `'False'`. Для `'store_false'` - соответственно все наоборот.

Использование параметров булева типа помогает при необходимости изменить поведение программы.

```

1 ► import argparse
2
3     parser = argparse.ArgumentParser(description='Description of your program')
4     parser.add_argument('-x', action='store_true')
5     parser.add_argument('-y', action='store_false')
6     args = parser.parse_args()
7     print(args)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py
Namespace(x=False, y=True)
(venv) root@exp:~/PycharmProjects/education# python lesson.py -x -y
Namespace(x=True, y=False)
(venv) root@exp:~/PycharmProjects/education#

```

Параметр `required=True` делает использование опционального аргумента обязательным.

```

1 ► import argparse
2
3     parser = argparse.ArgumentParser(description='Description of your program')
4     parser.add_argument('-x', action='store_const', const='Hello Admin!',
5                           required=True)
6     args = parser.parse_args()
7     print(args)
8     print(args.x)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py
usage: lesson.py [-h] -x
lesson.py: error: the following arguments are required: -x
(venv) root@exp:~/PycharmProjects/education# python lesson.py -x
Namespace(x='Hello Admin!')
Hello Admin!

```

По умолчанию количество значений аргумента равно 1. Но можно задать любое, определив их число через параметр **nargs** при добавлении аргумента.

Если количество значений для аргумента известно и фиксировано - в nargs указываем соответствующее число.

Если количество аргументов неизвестно, и может быть один и более аргумент - задаем **nargs='*'** со звёздочкой.

```

1 ► import argparse
2
3     parser = argparse.ArgumentParser()
4     parser.add_argument('-x', nargs=2)
5     parser.add_argument('-y', nargs='*')
6     args = parser.parse_args()
7     print(args)
8     print(args.x)
9     print(args.y)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py -x 5 6 -y 'cat' 8 'wood'
Namespace(x=['5', '6'], y=['cat', '8', 'wood'])
['5', '6']
['cat', '8', 'wood']

```

Особенности использования nargs:

- * Значения аргументов помещаются в список.
- * Нельзя использовать более одного псевдонима, то есть короткие и длинные имена одновременно.

Метод `parse_args()` позволяет не полностью указывать имя аргумента, а только его часть. Если при этом не возникает конфликта имен.


```

1 ► import argparse
2
3     parser = argparse.ArgumentParser()
4     parser.add_argument('-argument', nargs='*')
5     args = parser.parse_args()
6     print(args)
7     print(args.argument)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py -argument 17
Namespace(argument=['17'])
['17']
(venv) root@exp:~/PycharmProjects/education# python lesson.py -arg 17
Namespace(argument=['17'])
['17']
(venv) root@exp:~/PycharmProjects/education# python lesson.py -a 17
Namespace(argument=['17'])
['17']

```

Пример конфликта:

```

1 ► import argparse
2
3     parser = argparse.ArgumentParser()
4     parser.add_argument('-argument', nargs=1)
5     parser.add_argument('-a', nargs=1)
6     args = parser.parse_args()
7     print(args)
8     print(args.argument)
9     print(args.a)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py -argument 17 -a 26
Namespace(a=['26'], argument=['17'])
['17']
['26']
(venv) root@exp:~/PycharmProjects/education# python lesson.py -arg 17 -a 26
usage: lesson.py [-h] [-argument ARGUMENT] [-a A]
lesson.py: error: ambiguous option: -arg could match -argument, -a

```

Параметр `choices=` явным образом указывает, какие значения будут возможны для использования. Все необходимые значения указываются в виде списка.

```

1  ▶ import argparse
2
3  parser = argparse.ArgumentParser()
4  parser.add_argument('-a', choices=['a', 'b', 'c'])
5  args = parser.parse_args()
6  print(args)
7  print(args.a)

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python lesson.py -a 'a'
Namespace(a='a')
a
(venv) root@exp:~/PycharmProjects/education# python lesson.py -a 'x'
usage: lesson.py [-h] [-a {a,b,c}]
lesson.py: error: argument -a: invalid choice: 'x' (choose from 'a', 'b', 'c')

```

Модуль click

Click взаимодействует с командной строкой, как и argparse, но по-другому. Он использует декораторы, поэтому ваши команды должны работать обязательно с функциями, которые можно обернуть этими декораторами.

Простой пример – функция main выполняет всего 1 действие, выводит текст в консоль с помощью метода `echo()`. Функция становится инструментом командной строки Click, если его декорировать `click.command()`.

```

import click

@click.command()
def main():
    click.echo("Bla-bla-bla")

if __name__ == "__main__":
    main()

```

new ×
Bla-bla-bla

Мы можем получить справку используя стандартный `–help`.

```
(venv) root@exp:~/PycharmProjects/education# python new.py --help
Usage: new.py [OPTIONS]

Options:
  --help  Show this message and exit.
```

Для добавления опциональных параметров используйте `option()` и для позиционных `argument()` декораторы.

```
1 ▶ import click
2
3
4 @click.command()
5 @click.option('--count', default=50, help='number of repeats')
6 @click.argument('name')
7 def hello(count, name):
8     s = name*count
9     click.echo(s)
10
11
12 if __name__ == "__main__":
13     hello()
```

```
Terminal: Local x +  
(venv) root@exp:~/PycharmProjects/education# python new.py '*' --count 35  
*****  
(venv) root@exp:~/PycharmProjects/education# python new.py v --count 35  
vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv
```

Обратите внимание – при использовании в качестве параметра спецсимволов, их необходимо заключать в кавычки, тогда как слово/буква или цифра этого не требует.

В справке добавились наши опции.

```
(venv) root@exp:~/PycharmProjects/education# python new.py --help
Usage: new.py [OPTIONS] NAME

Options:
  --count INTEGER  number of repeats
  --help           Show this message and exit.
```

Добавим ещё один позиционный аргумент `title`, получится заготовка для вывода любых заголовков.

Обратите внимание – фразу нужно брать в кавычки, иначе количество аргументов будет неверным. И не забывайте, что позиционные аргументы идут в порядке очереди. Здесь первый name, а уже потом title. Опциональный аргумент можно вставлять в любом месте.

```

1  ▶ import click
2
3
4  @click.command()
5  @click.option('--count', default=50, help='number of repeats')
6  @click.argument('name')
7  @click.argument('title')
8  def hello(count, title, name):
9      s = f"{name*count}\n{title}\n{name*count}"
10     click.echo(s)
11
12
13 if __name__ == "__main__":
14     hello()

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python new.py '-' 'Каталог товаров:' --count 16
-----
Каталог товаров:
-----

```

Очень круто, что модуль имеет поддержку цветов методом `style()`, поэтому нет необходимости подключать дополнительные библиотеки.

```

1  ▶ import click
2
3
4  @click.command()
5  @click.option('--count', default=50, help='number of repeats')
6  @click.argument('name')
7  @click.argument('title')
8  def hello(count, title, name):
9      s = f"{name*count}\n{title}\n{name*count}"
10     click.echo(click.style(s, fg='green', bold=True))
11
12
13 if __name__ == "__main__":
14     hello()

```

Terminal: Local × +

```

(venv) root@exp:~/PycharmProjects/education# python new.py '-' 'Каталог товаров:' --count 20
-----
Каталог товаров:
-----

```

Параметры стилей:

- `text` - строка стиля с кодами ANSI.
- `fg` - если предоставлено, это станет основным цветом.
- `bg` - если предусмотрено, это станет фоновым цветом.
- `bold` - если предусмотрено, это включит или отключит полужирный режим.
- `dim` - если предусмотрено, это включит или отключит режим dim (берёт цветовой параметр из предыдущей строки). Это плохо поддерживается.

- **underline** - если это предусмотрено, это включает или отключает подчеркивание.
- **blink** - если предусмотрено, это включит или отключит мигание.
- **reverse** - если предоставлено, это включит или отключит инверсный рендеринг (передний план становится фоновым, а наоборот).
- **reset** - по умолчанию в конце строки добавлен код `reset-all`, что означает, что стили не переносятся. Это может быть отключено для создания стилей.

Поддерживаемые названия цветов:

- black (может быть серый)
- red
- green
- yellow (может быть апельсин)
- blue
- magenta
- cyan
- white (может быть светло-серым)
- bright_black
- bright_red
- bright_green
- bright_yellow
- bright_blue
- bright_magenta
- bright_cyan
- bright_white
- reset (сбросить только код цвета)

Метод **secho()** объединяет **echo()** и **style()** в один вызов. Таким образом, следующие два вызова одинаковы:

```
click.echo(click.style(s, fg='green', bold=True))
click.secho(s, fg='green', bold=True)
```

Метод **version_option()** предназначен для вывода версии и названия программы. При запуске программы с параметром **--version**, будет показана информация, и программа завершится.

version – номер версии

prog_name – название программы


```
1  ▶ import click
2
3
4  @click.version_option(version='1.0', prog_name='The header template ')
5  @click.command()
6  @click.option('--count', default=50, help='number of repeats')
7  @click.argument('name')
8  @click.argument('title')
9  def hello(count, title, name):
10     s = f"{name*count}\n{title}\n{name*count}"
11     click.secho(s, fg='green', bold=True)
12
13
14  if __name__ == "__main__":
15     hello()
```

Terminal: Local × +

```
(venv) root@exp:~/PycharmProjects/education# python new.py --version
The header template , version 1.0
```